# MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts inference must deliver selected experts even when the expert pool exceeds the memory assigned to a task. SSD reads then enter the decode path. MECC combines profile-guided placement, cross-tier retention, and dependency-aware transfers across HBM, pinned host memory, and SSD. Its design preserves FP16 weights and the router's selections. Across four Mixtral model/workload combinations, the single-GPU configuration provides 3.00–3.26× speedup over unoptimized streaming and 1.25–1.42× speedup over a matched SSD-native adapter. On Mixtral-8x7B question answering, it uses one GPU at 8.0 s/token; a two-GPU resident configuration reaches 2.8 s/token. The comparison exposes the latency and allocation choices of SSD-active MoE serving.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

## I. Introduction

Mixture-of-experts (MoE) models select a subset of experts for each token [1]–[3]. Mixtral and DeepSeekMoE instantiate sparse routing in language models [4], [5]. A single GPU can execute selected experts without full HBM residency, provided each weight is delivered before computation.

Shared servers assign each service a host-memory allowance. When combined HBM and host residency cannot hold the expert pool, requests reach SSD. FlexGen and DeepSpeed Inference already plan heterogeneous-memory execution [6], [7]; flash-aware inference also addresses models exceeding DRAM [8]. We study expert delivery under fixed task caps.

Prior work uses CPU/GPU orchestration [9], memory-constrained planning [10], and SSD-native delivery [11]. MoE-Infinity traces activations for cache replacement and prefetch [12]. ProMoE predicts future expert use [13]. Learned replacement [14] and domain-guided residency [15] further exploit routes. We compare placement, retention, and scheduling under common physical caps.

Three coupled decisions govern expert delivery. **First, capacity has to be distributed across layers.** Uniform slots can leave some layers repeatedly reading from SSD. **Second, HBM eviction has to preserve later reuse.** A fixed CPU pool may discard useful experts, while independent tier caches can retain duplicate copies. **Third, residency has to track request changes.** An initial profile covers startup demand; online observations update that placement as the working set changes.

Our design treats the route profile as an initial residency prior and actual requests as online corrections. MECC assigns these roles across HBM, CPU pinned memory, and SSD. The compiler assigns expert rankings and layer capacities. The runtime promotes requested experts and demotes HBM victims into the host working set. Transfer overlap requires independent work and enforced readiness before computation. Section III presents the framework; Section V defines physical accounting and metrics.

We evaluate the contribution of profile placement, retention, and transfer scheduling separately. We also compare one-GPU latency with the smallest feasible resident GPU allocation.

## II. Deployment Model and Challenges

### A. Single-GPU Execution with Bounded Host Memory

Consider $L$ MoE layers. Layer $\ell$ contains $N_\ell$ experts and selects $k_\ell$ per token. Expert $e = (\ell, j)$ is an immutable weight object of size $s_e$. The GPU also stores dense weights, activations, the KV cache, and temporary buffers. The expert-residency allowance is $B_{\mathrm{H}}$; the total task-level host allowance is $B_{\mathrm{C}}$. Host mirrors and staging buffers count toward $B_{\mathrm{C}}$.

Let $\bar{B}_{\mathrm{H}}$ and $\bar{B}_{\mathrm{C}}$ denote unique expert-residency capacities after charging other allocations. The target regime satisfies $\sum_e s_e > \bar{B}_{\mathrm{H}} + \bar{B}_{\mathrm{C}}$ and exceeds the task host allowance. Post-initialization SSD demand traffic measures its operational impact; startup reads alone do not establish this regime. Models that fit in host memory serve as two-tier controls. Models that fit in HBM need no expert offloading.

An HBM hit uses resident weights. A CPU hit requires host-to-device (H2D) transfer. An SSD miss adds an SSD-to-host read. We record the expert's tier when demand arrives, before servicing the request. Separate counters record physical SSD and transfer traffic, including prefetches.

### B. Challenge 1: Allocate Capacity across Layers

The value of a cache slot depends on the layer's request distribution and miss cost. A small working set needs few slots. A broader working set can cause repeated cold reads under the same allocation. This difference motivates moving capacity between layers while holding total memory fixed.

Figure 1(a) compares four representative layers sharing 16 equal-size CPU slots. Uniform allocation assigns four slots per layer; profile-guided allocation assigns two, three, five, and six. HBM capacity and replacement are fixed. The plan moves slots toward layers with greater SSD demand. The first two layers lose coverage while the last two gain it. Aggregate miss counts weight each layer by its request volume.

### C. Challenge 2: Preserve Reuse after HBM Eviction

An expert can be requested again soon after HBM eviction. A fixed CPU pool retains only its predefined members. Independent tier caches may instead retain copies of the same expert. These policies differ in both reuse coverage and physical duplication.

Figure 1(b) illustrates a fixed CPU pool, independent LRU, and MECC retention. These names identify retention rules, not distinct published implementations. Figure 3 shows the resulting promotions and demotions. Capacity-matched exclusive LRU is the direct reference for joint retention.

### D. Challenge 3: Initialize Residency and Track Changes

A profile supplies access information before the first request. Online history tracks changes after service begins. Fixed placement can become stale when the working set shifts. An online policy without a profile first has to discover that working set. Their combination targets startup misses and later adaptation.

Figure 1(c) compares profile initialization, online adaptation, and their combination across a shift in expert popularity. The contrast separates initial coverage from later correction.

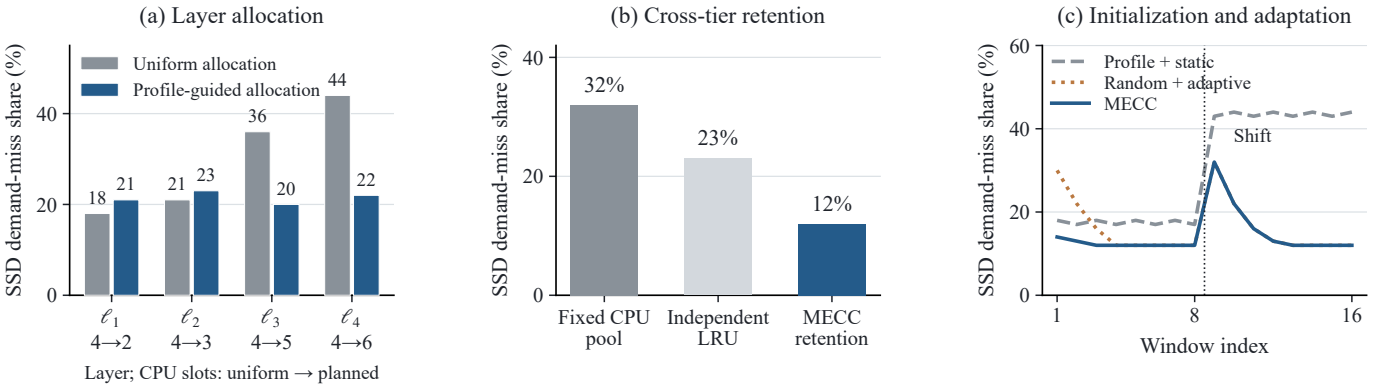



Fig. 1. Deployment challenges. (a) Four representative layers share 16 equal-size CPU slots; tick labels show uniform → planned slots. (b) Fixed CPU pool, independent LRU, and MECC retention under the same profile. (c) Profile + static, random + adaptive, and profile + adaptive (MECC) across a shift. Random + static is not plotted.

## III. Overview

Figure 2 separates offline compilation from online execution. An independent route profile, model metadata, task memory caps, and delivery costs define the compilation inputs. The output specifies layer capacities and initial expert IDs. Mirrors and staging are charged outside unique primary residency; SSD retains all backing weights.

The unchanged router supplies demand IDs. Residency lookup selects a source tier; the runtime promotes requested experts and demotes HBM victims. The delivery path enforces weight readiness before execution. Layer capacities stay fixed during a run, while expert identities and recency change. Planner controls therefore fix online rules; replacement controls fix quotas and initialization (Section VI-D).

## IV. Method

### A. Compiler: Profile-Guided Placement

The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability $p_e$ for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Here $q_{\ell,r}$ is conditional on an expert selection in layer $\ell$. Let $w_\ell$ be that layer's fraction of all selections. The global probability is $p_e = w_\ell q_{\ell,r}$, with $\sum_\ell w_\ell = 1$. For fixed top-$k$ routing, $w_\ell = k_\ell / \sum_j k_j$. Each rank retains its profiled expert ID. Empirical probabilities can replace the fitted Zipf distribution. Neither representation encodes request order.

Let $x_{e,t} \in \{0,1\}$ indicate the primary tier of expert $e$. The tiers are HBM, CPU, and SSD, denoted by H, C, and S. Residency budgets $\bar{B}_t$ must exclude buffers and mirrors. The following static objective is a reference model, not a claim that the current planner solves the full problem:

$$\begin{aligned} \min_{x} \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \bar{B}_t \quad (t \in \{\mathrm{H}, \mathrm{C}\}). \end{aligned} \tag{2}$$

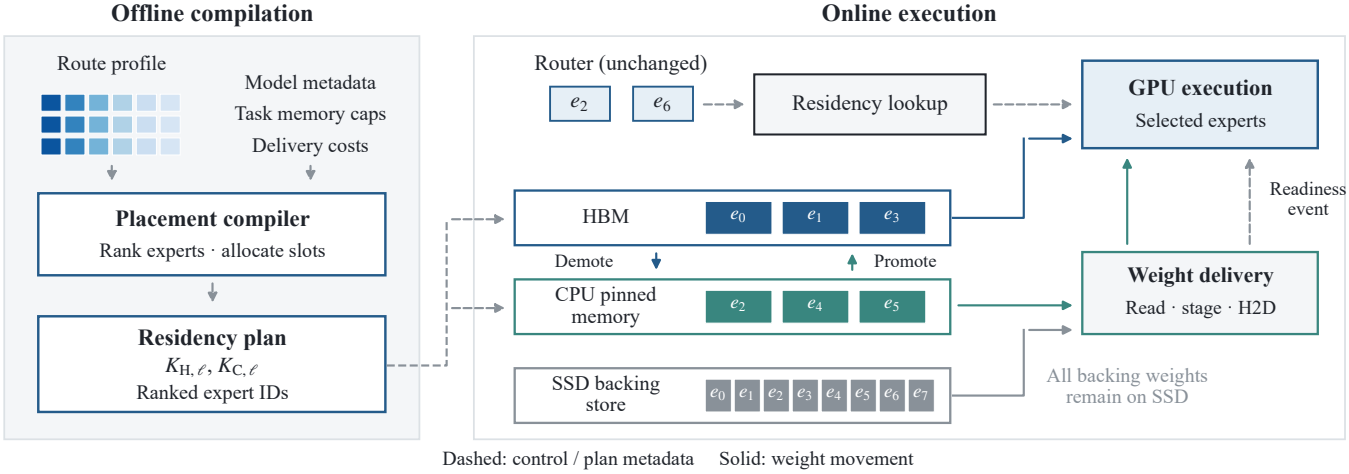



Fig. 2. MECC framework. Offline compilation emits capacities $K_{\mathrm{H},\ell}$ and $K_{\mathrm{C},\ell}$ and ranked expert IDs. Online lookup resolves unchanged router selections against HBM, CPU pinned memory, and SSD. Weight delivery records a readiness event before GPU execution. Dashed arrows carry control/plan metadata; solid arrows move weights. Expert IDs and occupancy are schematic.

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

*Proposition 1:* Assume equal-size objects, fixed request probabilities, independent tier capacities, and $T_H \le T_C \le T_S$. An optimal static placement fills HBM with the most probable objects, CPU with the next most probable objects, and SSD with the remainder.

*Proof sketch.* Suppose $p_i > p_j$ and object $i$ occupies the slower tier. Exchanging the two objects changes expected cost by $(p_i - p_j)(T_{\text{fast}} - T_{\text{slow}}) \le 0$. Repeated exchanges produce ranked placement. Filling an empty faster-tier slot also cannot increase cost. The result applies to the stated static model.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank $r$ adds

$$\Delta_\ell(r) = w_\ell q_{\ell,r}. \tag{3}$$

This is a coverage score, not the exact latency marginal of Equation 2. Tier-boundary movements can affect both HBM and CPU assignments. The heuristic allocates HBM slots first, then CPU slots from the remaining ranks. At each step it selects the largest next-rank score; ties use layer ID. Expert-ranking ties use expert ID. It stops at the tier's slot cap or exhaustion of experts. Per-layer minima, if enabled, are assigned first and an infeasible minimum allocation must be rejected. A direct scan costs $O(L(S_H + S_C))$ after ranking, where $S_t$ is a total slot cap. These rules define the compiler interface. Batched prefill additionally requires distinguishing token selections from distinct expert fetches. The sequential heuristic does not solve the general static objective.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. Input model/profile hashes, object sizes, and the ranking mode identify its provenance. A missing profile selects the natural-order fallback; the plan records the ranking mode. Initialization traffic and planning time are charged before comparing finite service horizons.

### B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An SSD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The SSD backing weights remain available.

Figure 3 starts with HBM $[e_0, e_1]$ and CPU $[e_2, e_3, e_4]$. Requesting $e_2$ promotes it and demotes $e_0$. The next request for $e_0$ hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

For a serial demand stream within one layer, concatenate the HBM MRU-to-LRU stack with the CPU MRU-to-LRU stack. Exclusive promotion and victim demotion preserve the corresponding LRU stack of capacity $K_H + K_C$. This reference requires matching initialization, no prefetch or alternative admission, and completed transitions before the next request. It does not imply equivalence to a cache that reallocates capacity across layers, or dominance over static placement.

### C. Delivery: Dependency-Aware Overlap

A correct asynchronous implementation requires a readiness event after each pinned-host copy. The computation stream must wait before consuming the weight. The source buffer must remain live until copy completion, and the resident weight until its last consumer finishes. These are required lifetime rules, including across separate loading and dispatch iterators; the execution stream must enforce them with readiness events.

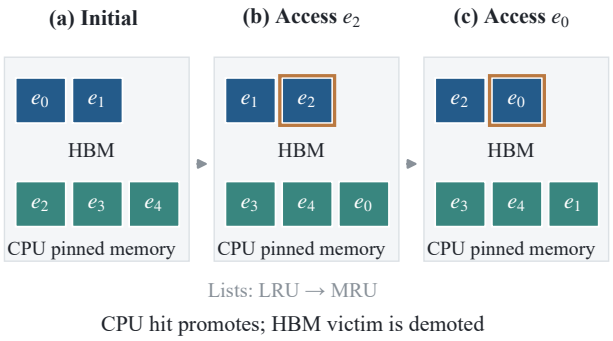



Fig. 3. Cross-tier retention with two HBM slots and three CPU slots. Lists run from LRU to MRU. A CPU hit promotes the requested expert and makes room for the HBM victim. Mirrors and staging consume additional physical space. Schematic.

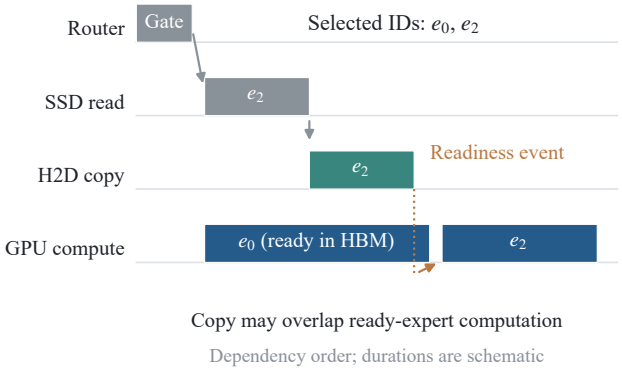



Fig. 4. Delivery dependencies after a gate selects $e_0$ and $e_2$. The resident $e_0$ can execute while $e_2$ is read and copied. The readiness event protects consumption of $e_2$. Kernel order is unchanged. Durations are schematic; the figure asserts no measured overlap.

At batch one, routing reveals the selected IDs only after the current layer's gate. Copying one selected expert may overlap computation of another already-ready expert if kernel and aggregation order permit. This provides no knowledge of a later layer's route. In-flight weights must be protected from eviction. Replacement comparisons must disable predictive prefetch; the overlap control must hold admission, residency, and requests fixed. The available overlap and its exposed critical path remain measurements, rather than an assumed gain from a copy stream.

Figure 4 pairs one HBM-resident expert with one SSD-backed expert from the same gate. SSD read precedes H2D and readiness. Copy completion releases the pinned source; final-consumer completion permits GPU eviction. Useful independent computation determines the available overlap.

The design requires unchanged weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. The validation protocol compares synchronous and asynchronous paths on identical inputs using weight

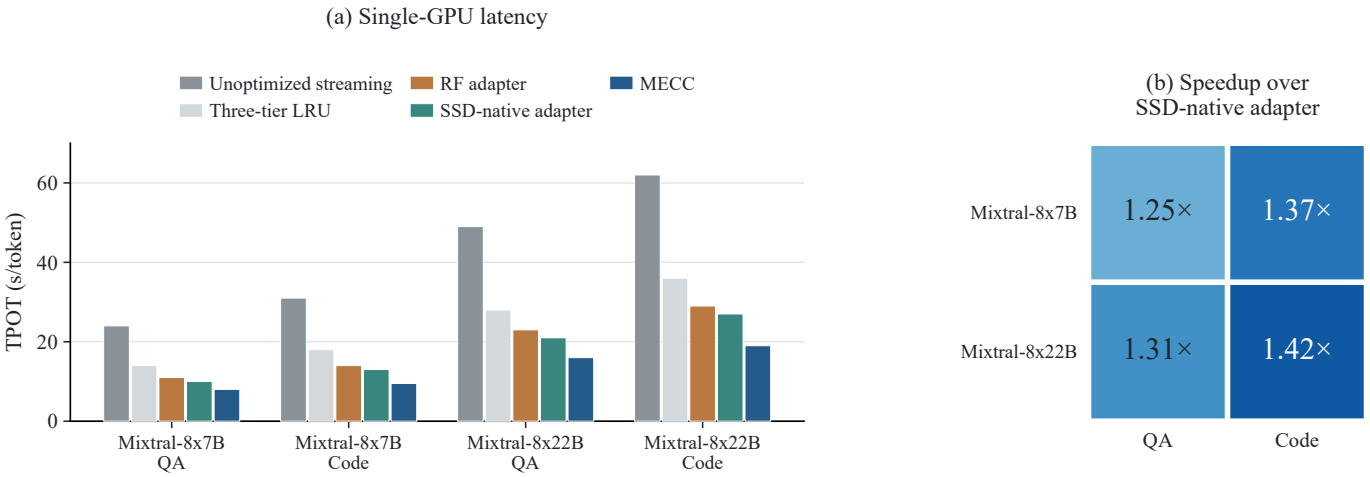



Fig. 5. Performance. (a) TPOT for single-A100 policies. RF adapter and SSD-native adapter are local mechanisms, not published-system measurements. Nominal allowances are 50 GB for experts and 40 GB for host memory; mirrors and staging belong to the task host cap. (b) Speedup over SSD-native adapter, calculated from the same TPOT targets. Plots show point estimates.

hashes, route selections, logits, tokens, and perplexity; the same protocol isolates transfer scheduling from changes in numerical execution.

## V. Implementation

Model adapters map layer/expert IDs to weight objects, and policies share an end-to-end loader. The harness maps each layer/expert ID to an immutable FP16 weight object. Policies share the same loader and router interface. Memory accounting includes physical mirrors and staging buffers.

Physical accounting deduplicates storage identities, not expert IDs: distinct live copies still consume bytes. Let $R_t$ be resident expert bytes, $D_{\mathrm{H}}$ distinct host mirrors, $S_t$ staging, and $O_t$ other charged allocations. The required constraints are

$$R_{\mathrm{C}} + D_{\mathrm{H}} + S_{\mathrm{C}} + O_{\mathrm{C}} \le B_{\mathrm{C}}, \tag{4}$$

$$R_{\mathrm{H}} + W_{\mathrm{dense}} + W_{\mathrm{KV/act}} + S_{\mathrm{H}} + O_{\mathrm{H}} \le B_{\mathrm{GPU}}. \tag{5}$$

Full mirroring requires $D_{\mathrm{H}} \ge R_{\mathrm{H}}$. Residency must be reduced or a mirror-free path validated. Peak measurements must include initialization and transient copies, not only steady residency.

Counters must separate initialization, warmup, prefill, decode, and quality evaluation. TPOT divides decode time by measured intervals; TTFT includes prefill and the first output. Demand-miss share is the fraction of selected-expert arrivals without ready HBM/CPU weights. In-flight prefetch is logged separately. Physical SSD/H2D/D2H bytes use the same phase boundaries. These counters define the same phase boundaries for every policy.

## VI. Evaluation

### A. Experimental Methodology

**Models and workloads.** The evaluation uses Mixtral-8x7B and Mixtral-8x22B [4], [16] with question-answering and code prompts. One A100 provides 80 GB of physical HBM. The nominal expert allowance is 50 GB; dense weights, KV cache, and staging consume additional HBM. The task host cap is 40 GB. Profile and evaluation splits are independent. Main comparisons use batch one, 128 input tokens, 128 output tokens, and deterministic decoding.

**Unoptimized streaming.** This baseline executes the same selected FP16 experts on one GPU, sharing the model, tokenizer, kernels, SSD-read semantics, and task caps with MECC. Demand staging disables profile placement, dynamic CPU retention, and overlap. Report its default residency and achieved occupancy; equal caps alone do not establish an equally provisioned comparison.

**Cache baselines.** Three-tier LRU controls for basic caching. RF and SSD-native are local mechanism adapters. The recency/frequency (RF) adapter uses a heuristic with recency and frequency scores; FlashMoE [14] learns a predictor from these signals. SSD-native represents three-tier residency motivated by SSD-LLaMA [11]. Neither establishes a reproduction of the cited system. External systems are discussed separately in Section VII.

**Multi-GPU reference.** The resident reference partitions the full FP16 model across two, four, or eight A100s where capacity permits. It uses matched inputs and decoding without SSD delivery. Its resource record includes GPU count, parallelism, interconnect, software revision, and memory peak. The resident reference is a capacity and latency comparison.

**Metrics.** We report TPOT, TTFT, throughput, demand-miss share, SSD/H2D/D2H bytes, and physical memory peaks. GPU-time comparisons use the same workload. The charts report deterministic point estimates under the stated workload.

### B. Latency under Matched Single-GPU Budgets

Figure 5(a) fixes one A100 and the task host cap across four model/workload combinations. TPOT is 8.0, 9.5, 16.0, and 19.0 s for MECC, versus 24, 31, 49, and 62 s for unoptimized streaming. These values correspond to 3.00×, 3.26×, 3.06×, and 3.26× speedup. Section VI-D decomposes the complete stack.

SSD-native adapter takes 10, 13, 21, and 27 s per token. Figure 5(b) gives the corresponding 1.25–1.42× gains. This is the closer local cache comparison; the larger streaming gain does not establish an advantage over other cache systems.

TTFT is 60/68/96/112 s for MECC and 130/155/230/280 s for streaming. These values describe first response. TPOT describes later token intervals. Initialization is charged separately in the cost analysis below.

### C. Latency versus GPU Allocation

The resident reference has eight-GPU TPOT of 1.0/1.2/1.8/2.1 s in the same workload order. Figure 7(b) and Table I include the smaller

TABLE I
MIXTRAL-8X7B QUESTION ANSWERING: LATENCY AND GPU ALLOCATION.

| Configuration | TPOT (s) | GPU-s/token |
|---|---|---|
| MECC, 1 GPU | 8.0 | 8.0 |
| Resident, 2 GPUs | 2.8 | 5.6 |
| Resident, 4 GPUs | 1.65 | 6.6 |
| Resident, 8 GPUs | 1.0 | 8.0 |

TABLE II
COMPONENT CONTROLS. EACH ROW CHANGES ONE MECHANISM.

| Component | Vary | Hold fixed |
|---|---|---|
| Allocation | Layer quotas | Ranking, online policy, bytes |
| Initialization | Initial expert IDs | Quotas, requests, online policy |
| Retention | Replacement rule | Quotas, initial stack, admission |
| Overlap | Schedule | Requests, transfers, kernel order |

resident allocations. The single-GPU configuration serves workloads with a declared latency allowance.

Table I includes one-, two-, four-, and eight-GPU configurations. The two-GPU resident configuration is faster and uses fewer nominal GPU-seconds than single-GPU MECC. A single-GPU deployment instead serves offline generation when GPU availability is the binding resource. For 128 output tokens, 127 decode intervals at 8 s/token require about 17 minutes.

### D. Component and Mechanism Analysis

Figure 6 adds components to streaming on the first workload. Profile placement reduces TPOT from 24 to 18 s. Retention reduces it to 11 s. Overlap reduces it to 8 s. Each change measures a component within that stack. Component interactions require additional leave-one-out and factorial comparisons.

SSD demand-miss shares are 46%, 33%, 20%, and 20%. Placement and retention reduce the illustrated SSD demand share. Overlap keeps the same miss share and reduces exposed waiting. The 12% point in Figure 1(b) belongs to a separate controlled trace.

Mechanism controls hold requests fixed. Separate controls must vary layer allocation and initial expert identities while holding the other fixed, and charge initial transfers. The retention comparison must record duplicate bytes under each policy. Within-phase shuffling preserves expert histograms and tests sensitivity to order. The shift workload tests adaptation to a changed expert distribution.

Table II defines the comparisons needed for attribution. An allocation experiment redistributes quotas but retains the same ranking rule. An initialization experiment changes expert IDs within fixed quotas. The retention control uses identical primary capacities and initial stacks. The overlap control must preserve physical transfers as well as demand misses; equal miss percentages alone do not rule out extra I/O. These controls isolate the four mechanisms.

### E. Resource Sensitivity, Cost, and Quality

Figure 7(a) increases the host cap from 16 to 48 GB on a fixed GPU. TPOT falls from 16 to 7 s for MECC and from 27 to 12 s for conventional LRU. The paired curves compare residency at each task allowance. The task host cap includes mirrors and staging buffers.

Figure 7(b) varies GPU count for 8x7B question answering. The resident configuration does not fit on one GPU. Its two-, four-, and eight-GPU latencies are 2.8, 1.65, and 1.0 s/token. Only the single-GPU MECC configuration is shown. The one-GPU result is compared with the smallest feasible resident allocation.

Figure 7(c) adds initialization to decode time. Initialization costs 30 s for MECC and 12 s for the SSD-native adapter. Decode costs 8 and 10 s/token, respectively. The plotted terms are equal at nine decode intervals after the first token. This is not a full-request crossover. For $n$ output tokens, full cost is $J(n) = I + \mathrm{TTFT} + (n-1)\mathrm{TPOT}$, with amortized profiling/loading included in $I$ and no double counting. The adapter TTFT and profile costs are unspecified, so no numerical full-request break-even follows.

Other sensitivity axes include HBM capacity, input/output lengths, batch size, and profile coverage or mismatch. Leave-one-out controls separate zero-copy eviction, pinning, batch H2D, and prefetching. A reverse resource comparison fixes a TPOT limit and finds each policy's minimum feasible host cap. The plots focus on host-cap, GPU-count, and service-horizon sensitivity.

The profile's service horizon matters. Its collection cost is charged once per frozen model/workload profile, then amortized over the requests that actually reuse it. A shorter horizon can favor the same runtime without profile initialization. Report planning time, initial SSD/H2D bytes, and request count before interpreting a steady-state TPOT gain. Cold-start and warm-service measurements answer different deployment questions.

The exactness protocol compares weight hashes, expert selections, logits against synchronous execution, output tokens, and perplexity on a fixed corpus. Nondeterministic kernels require prespecified error tolerances and token comparisons. Weight checks and execution checks test separate properties.

## VII. RELATED WORK

**Sparse execution and serving.** Switch and GShard decouple active computation from total parameters [2], [3]; DeepSeekMoE uses fine-grained and shared experts [5]. DeepSpeed-MoE optimizes distributed inference [17]; Tutel adapts parallelism/pipelining [18]; MegaBlocks targets sparse training kernels [19]. PagedAttention and SGLang optimize KV allocation and prefix reuse [20], [21], distinct from expert residency.

**Heterogeneous memory.** ZeRO-Infinity uses GPU/CPU/NVMe for training [22]; DeepSpeed Inference supports heterogeneous serving [7]. FlexGen plans tensor placement and throughput-oriented schedules [6]. LLM in a flash combines reuse windows with flash-friendly layouts [8]. PowerInfer splits hot/cold neuron computation across GPU/CPU [23].

**Offloading.** Fast MoE offloading and SwapMoE target memory-constrained serving [24], [25]. Fiddler uses popularity-based GPU placement and CPU expert computation [9]. MoE-Lightning searches batching, execution, and weight/KV placement under CPU/GPU constraints, excluding disk variables [10]. Klotski combines expert-aware multi-batch execution with constraint-sensitive planning and correlation-aware prefetching [26].

**Expert prediction.** Pre-gated MoE changes the gating design [27]; ProMoE predicts expert use from intermediate results [13]; MoE-Infinity uses activation traces for replacement/prefetch [12]. MECC's demand-only control excludes prediction; its overlap control uses IDs available from the unchanged router.

**SSD-based serving.** SSD-LLaMA combines expert-pack I/O, lossless transport, frequency/recency-based VRAM admission, and RAM LRU [11]. Transient experts may bypass retention. FlashMoE learns a replacement predictor from recency/frequency signals and Belady-derived labels [14]. Plain RF/LRU omits these mechanisms.

**Coordinated residency.** MoE-CORE combines layer hit-rate curves, domain initialization, and routing history/scores [15]; optional substitution is separate from exact execution. ZipMoE combines

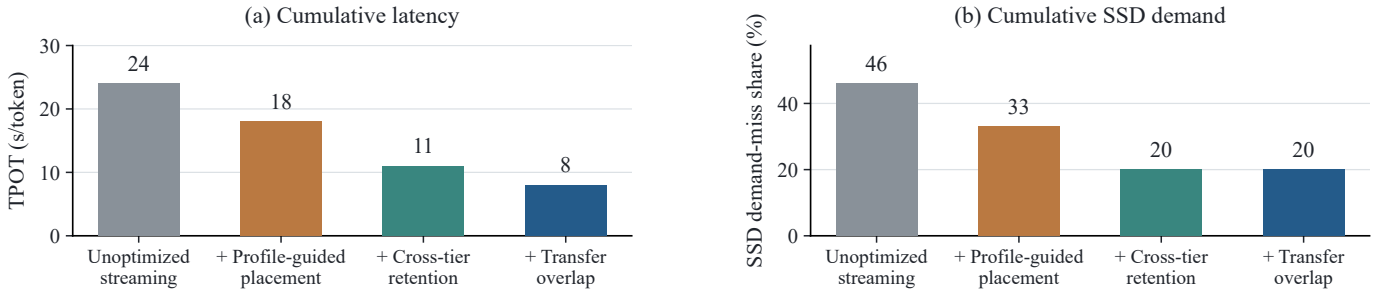



Fig. 6. Component ablation. (a) TPOT after adding each component. (b) SSD demand-miss share. The overlap-only step fixes admission, residency, and requests. Its miss share stays at 20%; exposed transfer time decreases.

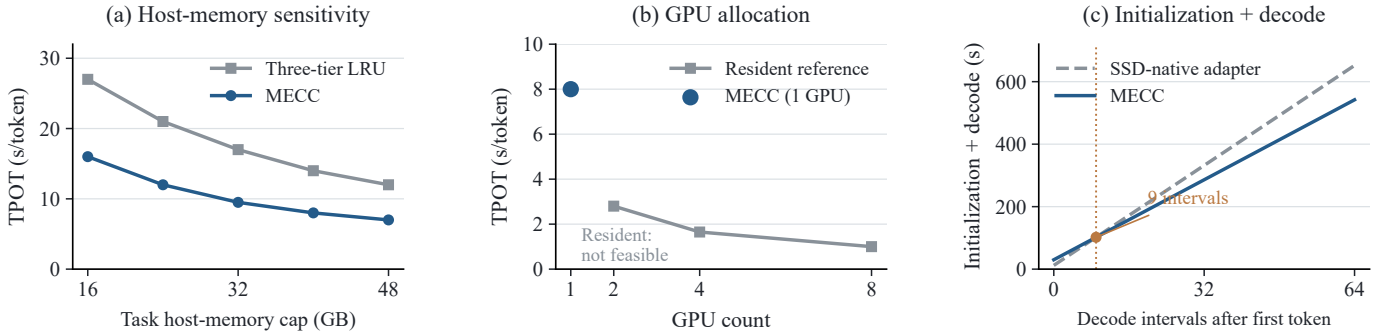



Fig. 7. Resource sensitivity. (a) TPOT versus total task host cap. (b) Allocation comparison on 8x7B question answering; the full-model-resident reference exceeds one GPU. Only the single-GPU MECC design is specified. (c) Initialization plus decode intervals. An 18 s initial overhead and a 2 s/interval saving yield equality at nine intervals. Additional loading, profiling, and prefill costs are outside this illustration.

lossless compression, compression-aware caching, and cache-affinity scheduling [28]. These overlaps require matched controls, not novelty claims based on module names.

The scope is SSD-active single-GPU serving. Local adapters do not establish superiority over these systems.

## VIII. Conclusion

MECC combines profile-guided placement, cross-tier retention, and readiness-checked FP16 delivery for SSD-active single-GPU inference. The evaluation distinguishes fixed-budget latency from the GPU-allocation tradeoff. The comparisons show the latency cost of constrained GPU allocation and the benefit of coordinating placement, retention, and delivery.

## References


[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[4] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[5] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[6] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[7] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[8] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[9] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[10] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[11] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[12] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[13] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[16] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[17] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[18] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[19] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[20] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[21] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[22] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[23] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[24] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[25] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[26] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[27] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[28] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.